YANG ZHAO

857-225-1826 • yangzhao200124@gmail.com • linkedin.com/in/yang-zhao-48b12431a • github.com/YZhao-prog

EDUCATION

Northeastern University — Boston, MA
Master of Science in Information Systems, GPA: 3.8/4.0 — Sep. 2024 – Dec. 2026 (Expected)
Relevant Coursework: Linux/UNIX Systems, Data Structures, Algorithms, Object-Oriented Design

The Chinese University of Hong Kong, Shenzhen — Shenzhen, China
Bachelor of Engineering in Computer Science and Engineering — Sep. 2020 – Jun. 2024
Relevant Coursework: Database Systems, Operating Systems, Computer Architecture, Compilers, Computer Networks

University of California, Berkeley
Summer Session (CS 61A: Structure and Interpretation of Computer Programs), GPA: 4.0/4.0 — May 2022 – Aug. 2022

PROFESSIONAL EXPERIENCE

Cisco (Splunk) — San Jose, CA
Backend Software Engineer Intern — Jun. 2026 – Sep. 2026

- Designed and implemented a **disk-based cache management system** in **C++** for a distributed search platform, introducing **size-aware cache eviction** to reclaim local storage, prevent disk-full failures, and preserve search result availability.
- Built an incremental **in-memory storage tracker**, replacing expensive **full-directory scans** with **O(1)** incremental disk usage accounting and eliminating repeated filesystem traversal.
- Integrated **AWS S3** with a **distributed metadata database** to track offloaded artifacts, enabling **safe local cache eviction** by separating local eviction from permanent cloud deletion and preserving recoverability until TTL expiration.
- Implemented configurable **per-search disk usage guardrails** that terminate **oversized in-progress searches** before they exhaust local storage, preventing runaway searches from destabilizing the cluster.

Cisco (Splunk) — Boston, MA
Backend Software Engineer Intern — Sep. 2025 – Dec. 2025

- Built cross-node artifact sharing system for Splunk's distributed search platform, enabling search results to persist and be accessed across **Search Head Cluster nodes** using **C++, AWS S3, REST API.**
- Owned the **end-to-end** design and rollout of a remote artifact offloading pipeline, driving production adoption across multiple search clusters.
- Optimized the artifact storage pipeline with batching and async uploads to sustain high-throughput workloads, enabling **horizontal scaling** by eliminating node-local state dependencies with only a **0.62%** increase in storage cost**.**
- Delivered **zero-downtime** cluster upgrades by designing **backward-compatible storage schema**, enabling **rolling migrations** across **multi-node clusters** without service interruption.
- Designed **Kubernetes** alerting strategy for artifact offloading workflows; implemented monitoring dashboards in **Splunk** to track pipeline health.
- Built a test suite using **pytest** and Splunk internal testing frameworks covering artifact lifecycle and **REST API** behavior, validated at production scale under **600K+** daily searches with **zero failures** and no runtime degradation, and integrated with **CI/CD** pipelines for automated validation.

PROJECTS

Distributed SQL Database | Rust, SQL, Redis, RocksDB

- Built a **distributed SQL database** in **Rust** with a query parser (**nom**), cost-based optimizer, and parallel execution engine, achieving **~3× throughput** over a single-threaded execution baseline on analytical queries.
- Implemented a hybrid storage layer using **RocksDB** and **Redis** caching, achieving ~50K QPS for point queries when benchmarked on an AWS c5.2xlarge instance.
- Designed **MVCC** transaction system with **snapshot isolation** and **optimistic concurrency control**, supporting high-contention workloads with strong consistency guarantees.
- Implemented custom **LSM-tree** storage engine in **Rust** with memtables, SSTables, leveled compaction, and Bloom filters for efficient range scans.

Distributed Key-Value Store with Raft Consensus | Golang, RPC, MapReduce

- Implemented **Raft consensus protocol** with leader election, log replication, persistence, snapshot compaction, and crash recovery, ensuring correct operation under network partitions.
- Built linearizable Key/Value service on **Raft**, supporting concurrent clients with request deduplication and at-most-once semantics, and validated correctness using linearizability testing.
- Developed **sharded Key/Value store** with replicated Shard Controller, supporting dynamic shard migration and rebalancing across Raft-backed replica groups while preserving consistency.
- Implemented **MapReduce** framework with coordinator and distributed workers, supporting parallel execution with **fault tolerance**.

TECHNICAL SKILLS

Languages: C++, Rust, Go, Java, Python, SQL, JavaScript/TypeScript, HTML/CSS
Backend: Spring Boot, Spring Cloud, Django, Node.js/Express, gRPC, Protocol Buffers, REST APIs
Frontend: React, Next.js, Angular
Databases & Messaging: MySQL, PostgreSQL, MongoDB, Redis, RocksDB, Elasticsearch, Kafka, RabbitMQ
Cloud & Infrastructure: AWS (EC2, S3, Lambda, RDS), Docker, Kubernetes, Helm, Jenkins, Git, Linux, Bash, CMake, CI/CD
Observability: Splunk, OpenTelemetry, Datadog, Prometheus, Grafana